ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Funciones en C

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase Activa: Módulo 1

Ejercicio 1.1

Cree una función, sin recibir parámetros ni retorna valores, que salude por pantalla al usuario. Utilícela en la función main().

Aprendemos: Declaración y uso de función, función sin retorno, tipo de dato void.

Clase Activa: Módulo 1

Ejercicio 1.2

Cree una función que solicite por pantalla al usuario un número entero y lo retorne. Utilícela en la función main(), almacenando dicho valor en una variable y mostrándola por pantalla.

Aprendemos: Declaración y uso de función, función con retorno, diferencia printf vs return.

Clase Activa: Módulo 2

Responda a las preguntas expuestas en los comentarios de los siguientes códigos. Esta actividad será resuelta en clases junto al profesor.

Aprendemos: Variables globales, variables locales, ámbito, ciclo de vida de una variable.

Clase Activa: Módulo 2

Ejercicio 2.1

```
#include<stdio.h>
#include<math.h> //para función pow

float potencia(float a, float b);

int main(){
        int x = 5, y = 6;
        float calculo = x * potencia(y, 2.5);
        // acá se hacen más cosas
        return 0;
}

// ¿Cuál es el problema del código?
```

Clase Activa: Módulo 2

Ejercicio 2.2

```
#include<stdio.h>
#include<math.h> //para función pow

int main(){
        int x = 5, y = 6;
        float calculo = x * potencia(y, 2.5);
        // acá se hacen más cosas
        return 0;
}

float potencia(float a, float b) {
        return pow(a,b);
}
// ¿Cuál es el problema del código?
```

Clase Activa: Módulo 2

Ejercicio 2.3

```
#include<stdio.h>
#include<math.h> //para función pow

float potencia(float a, float b) {
        return pow(a,b);
}

int main(){
        int x = 5, y = 6;
        int calculo = x * potencia(y, 2.5);
        // acá se hacen más cosas
        return 0;
}

// ¿Cuál es el problema del código?
```

Clase Activa: Módulo 2

Ejercicio 2.4

```
#include<stdio.h>
#define N 10

int main(){
        const int n = 5;
        n = 10; //¿Cómo actúa el compilador?
        printf("%d\n", n*N);

        return 0;

        // ¿Cuál es el problema del código?
        // ¿Qué diferencia a n de N?
}
```

Clase Activa: Módulo 2

Ejercicio 2.5

```
#include<stdio.h>

int main(){
        int i=10;

        if(i==10){
                int i = 5;
        }

        printf("El valor de i es: %d\n", i);
        // ¿Qué valor imprime?

        return 0;
}
```

Clase Activa: Módulo 2

Ejercicio 2.6

```
#include<stdio.h>

void saludo(){
        int i = 5;
        printf("Hola, numero %d", i);
}

int main(){
        int i = 10;
        saludo();
        printf("Hola, ¿qué tal?, numero %d\n", i);
        return 0;
}
// ¿Qué imprime este código?
```

Clase Activa: Módulo 2

Ejercicio 2.7

```
#include<stdio.h>

int a=1;
int b=2;

int main(){
        int a = 10;
        b++;
        printf("%d %d\n", a, b);
        return 0;
}

// ¿Qué imprime este código?
```

Clase Activa: Módulo 3

Ejercicio 3.1

Cree una función que reciba como parámetro una temperatura en grados Kelvin y retorne la temperatura en grados Celsius.

$$^{\circ}K = {}^{\circ}C + 273{,}15$$

Aprendemos: Parámetros, valor de retorno.

Clase Activa: Módulo 3

Ejercicio 3.2

Cree una función que reciba como parámetro una temperatura, su tipo (Celsius, Fahrenheit o Kelvin), y un tipo de temperatura de salida diferente. La función debe retornar la temperatura en la opción de salida ingresada.

$$^{\circ}C = \frac{5}{9} \times (^{\circ}F - 32)$$

Aprendemos: Parámetros, valor de retorno.

Clase Activa: Módulo 3

Ejercicio 3.3

Cree un programa que pregunte por pantalla un tipo de temperatura y un valor, y luego pregunte en que otra unidad quiere como transformarlo. (por ejemplo, inicia como Kelvin, y se entrega Celsius o Fahrenheit). **Debe usar las funciones creadas en el Ejercicio 3.1 y 3.2. Además, puede crear más funciones.**

$$^{\circ}F = \frac{9}{5} \times (^{\circ}K - 273{,}15) + 32$$

Aprendemos: Modularización, Uso de funciones, construcción basada en funciones.

Clase Activa: Módulo Desafío

Ejercicio Desafío

Cree una función que reciba como parámetro un día de la semana y retorne un número que represente a dicho día.

Aprendemos: Modularización, Uso de funciones, construcción basada en funciones.

Clase Activa: Módulo Desafío

Ejercicio Desafío

Cree una función que reciba como parámetro un número y retorne el nombre del día de la semana al que éste representa.

Aprendemos: Modularización, Uso de funciones, construcción basada en funciones.

Clase Activa: Módulo 4

Ejercicio Desafío

Cree un programa que pregunte un día de la semana y luego una cantidad X de días. El programa debe retornar el día de la semana que será en X días. **Debe utilizar las dos funciones diseñadas anteriormente.**

Aprendemos: Modularización, Uso de funciones, construcción basada en funciones.

Información de Vital Importancia para la Clase Activa[1].

[1] Depende exclusivamente de uds. leerla y comprenderla.

Funciones en C

- Encapsula código.
- Permite reutilizar código.
- Se pueden compilar por separado.
- Retorna siempre un tipo de dato específico.

Funciones en C

- Se puede separar interfaz (prototipo) de implementación.
- **Interfaz (prototipo):** Sólo define existencia y formato de función. Suficiente para ser invocada.

C

```
//tipo_retorno funcion(parametros con o sin nombre);
float potencia(float a, float b);
//....

calculo = x * potencia(y, 2.5);
```

Funciones en C

- **Implementación:** Define implementación de código de función.

C

```
/*tipo_retorno funcion(parametros con nombre) {
*       Implementacion...
}*/

float potencia(float a, float b) {
        return pow(a,b);
}
```

Funciones en C

- Si se utiliza solamente la implementación, está tiene que estar definida **antes** de la función que la utiliza.
- Si se utiliza el prototipo, es éste el que debe estar defindo **antes** de la función que la usa.
- Prototipo + Implementación: libertad de ubicación de la implementación $\rightarrow$ diferentes archivos.

Funciones en Python

- **Python** sólo permite implementación, sin prototipos.

Python

```
def nombre_funcion(parametros):
    #...Implementacion
    return <expresion>
```

```
def potencia(x, y):
    print x, y
    return x**y

y = 2
calculo = 5 * potencia(y, 2.5)
print calculo
```

```
2 2.5
28.2842712475
```

Macro

Una macro es un valor o conjuntos de instrucciones constantes que se definen a nivel de preprocesamiento. No es una variable y no puede ser modificada.

C

```
#include<stdio.h>
#define MACRO 10

int main(){
        int a = 5;
        printf("%"s\n", a*MACRO);
}
```

Constante

Una variable constante es aquella que se declara con el keyword **const**, e implica que ésta no puede ser modificada. Tiene todo el resto de las propiedades de una variable común.

C

```
#include<stdio.h>

int main(){
        const int a = 5;
        a = 10; //¿Cómo actúa el compilador?
        printf("%d\n", a);
}
```

Ámbito (Scope)

Las funciones (includa main) definen el alcance de las variables, llamado **ámbito**, vale decir, el segmento en el cual existen y pueden ser usadas → entre { y }.

C

```
#include<stdio.h>

void saludo(){ // var i sirve sólo dentro de función saludo.
        int i = 5;
        printf("Hola alumno numero %d", i);
}

int main(){// var a sirve sólo dentro de función main.
        int a = 10;
        saludo();
        printf("%d\n", a);
        return 0;
}
```

Variable local

Una variable local, es aquella que está definida dentro de un ámbito determinado, vale decir, entre llaves { y }.

Esto aplica para todo elemento que contenga llaves, incluida funciones y estructuras de control.

Variable local

C

```
#include<stdio.h>

int main(){// var a sirve sólo dentro de función main.
        int a = 10;

        if(a==10){
                int b = 1;
                printf("%d %d\n", a, b);
        }
        // ¿Qué pasa con la variable b?
        printf("%d %d\n", a, b);

        return 0;
}
```

Variable Global

Una variable global, es aquella que está definida **fuera de cualquier** ámbito.

Puede ser llamada, modificada y utilizada desde cualquier ámbito, siempre y cuando **no exista una variable local** con el mismo nombre.

Variable Global

C

```c
#include<stdio.h>

int a=1; //global
int b=2; //global

int main(){
        int a = 10; //local
        b++;
        printf("%d %d\n", a, b);//¿Qué imprime?
        return 0;
}
```

En python, no se pueden modificar variables globales dentro de una función.